

Driver Drowsiness Detector

Hardware Project Evaluation

Charles Smelt

F331073

April 3, 2026

Contents

1	Introduction	3
1.1	Why Do This?	3
2	Research	4
2.1	Tiredness Identifiers	4
2.2	Driving Effectuated Factors	4
2.3	Legality of Using a Mobile Device	4
2.4	Potential Models and Algorithms	5
2.4.1	Initial thought process – basic Convolutional Neural Netwrk (CNN):	5
2.4.2	Deep Convolutional Models:	5
2.4.3	Xception Model (Extreme Inception):	5
2.4.4	CondenseNet:	6
2.4.5	Further Development Ideas / Key Takeaways:	6
2.4.6	Information Communication:	7
3	Plan	9
3.1	Time Management:	9
3.2	Connections:	9
3.3	Component Choices and Reasoning:	10
4	Datasets	13
4.1	SIVAS-UNIVERSITY-OF-SCIENCE-AND-TECHNOLOGY-DRIVER-DROWSINESS-DATASET:	13
4.2	VICOMTECH:	13
4.3	FATIGUEVIEW:	13
4.4	BINA University:	13
4.5	DATASET POSITIVES AND NEGATIVES:	14
5	Development Checklist	16
6	Development	18
6.1	Setup:	18
6.2	Data Validation	19
6.3	Simple AI Models	20
6.3.1	Libraries	20
6.3.2	Simple AI Model	20
6.3.3	Inception Model	21
6.3.4	Dataset Resolution	22
7	connectivity	24
7.1	Back-End:	24
7.2	Front-End:	27
8	Evaluation	28

1 Introduction

The aim of this project is to create a versatile piece of hardware that can be used to improve road safety. With use of a Raspberry Pi 4B and a camera (I have opted for a web cam as they are cheap and widely accessible), I will be aiming to produce a light weight AI based system to detect a drivers tiredness and aim to either wake them up, warn them of their condition or even potentially come up with a viable solution. I will further aim to inform the driver through use of a wireless digital display. For this, I am aiming to use the users own WIFI enabled device to connect to the Pi and broadcast data, warning as and other information. Since this will be presented to the user via a web page, I would further like the user to be able to interact with the system to Tailor it to themselves and their set up, to get the best function out of the project.

1.1 Why Do This?

Many accidents on the road are caused by drivers lack attention due to dreariness. With ‘10-20%’ [1] of all crashes on the road believed to have been attributed to tiredness; one in four fatalities estimated to have had tiredness play a part on UK roads, this is a serious problem. Not only this but given that an incident occurs on UK roads ‘Every 17 minutes’ [2] where ‘someone is killed or seriously injured’ [2], something needs to change.

2 Research

2.1 Tiredness Identifiers

There are several different ways to identify tiredness, especially when driving. Some examples are:

- More frequent blinking – this is because your eyes and brain function to counter fatigue and discomfort
- Harder Blinking – prolonged opening can result in them being saw and dry, so, to combat this, your body makes you blink harder and more frequently to keep them lubricated
- Delayed reaction times – neurons in your brain slow as they process information less efficiently
- Your head can begin to drop as you inevitably slowly fall asleep (worst case)
- Yawning – this is your body's attempt to both keep your brain awake as well as cool the body
- Grip loosens – your body begins to relax as you fall asleep, actions aren't made with the same intent
- Mood changes – you can often become more irritable as you fall asleep
- Snoring – if you have already fallen asleep, you relax and this can result in snoring

2.2 Driving Effectuated Factors

As you get more tired, you often lose concentration. You can't find yourself reacting more slowly and your decision making can become impaired. You can also become a more violent or unstable driver as your mood can be easily influenced as you do not have the energy to combat how you feel. Not only this, but in the worst case (in the event of falling asleep) you can no longer operate the vehicle.

2.3 Legality of Using a Mobile Device

In this project, I would like to make use of the users' own device, which will probably be their mobile phone. To get around the legal issue of using a mobile device, it will have to be securely fastened in a cradle and not held by the driver. Furthermore, I will not be able to request interaction from the driver once the vehicle is in motion unless I use hands-free [6], "for example:

- A Bluetooth headset
- Voice commands
- A dashboard holder or mat
- A windscreen mount
- A built-in satnav"

Failure to comply with this can result in 6 penalty points and a £200 fine. But this can go up to a driving ban as well as a maximum of £1000 fine.

Key Implementation Takeaways:

- **Setup Protocol:** User-interactive features must be restricted to the initial setup phase while the vehicle is stationary.
- **Motion Safety:** Integration of the device's internal accelerometer to disable interactive displays during motion, ensuring the system does not become a distraction.

2.4 Potential Models and Algorithms

2.4.1 Initial thought process – basic Convolutional Neural Netwrk (CNN):

My initial idea was to follow a methodology I had previously learnt in one of my prior modules, using a simple Convolutional model consisting of input nodes containing extracted values from an image, and then train the model to detect these trends in data to predict an outcome. For this I made use of Mediapipe's face mesh to extract features from frames extracted from videos, to form inputs into the neural network. This was my initial thought process, and with a desire to start coding, and deliver a prototype, I naively started to develop this without additional research.

This methodology and implementation is talked about in 'Simple AI Models' section of this document. Here I explain my methodology and thought process (as well as data validation) as to why I believed this method feasible in the early stages of this project.

Following this failure, which is also expressed in the 'Simple AI Models' section I expanded my research to other AI models which will follow.

2.4.2 Deep Convolutional Models:

As an expansion from the idea above, this consists of a far greater number of layers. With its main uses in image identification, this would be ideal for the task at hand. One key difference being the number of layers. This adds size and complexity to the network, which may make it more difficult to run on the Pi compared to other models due to its increased parameter size as a result of its increases in width and levels.

2.4.3 Xception Model (Extreme Inception):

A development of the normal inception model, this stands as the middle ground between regular convolutional models and depthwise separable convolutional models (depthwise + pointwise). This makes better use of parameters than the Inception v3 model rather than reducing the number of parameters. This is done through channel mixing.

By removing the multi branch capabilities of an inception model, opting for a single branch this completely decouple spatial correlations as well as branch correlations, which results in more focused learning .

To take this further, you can make use of pre-trained models. The example I have used in the Inception Models section of this paper was trained on the ImageNet dataset.

This transferred learning allows for pre-learns identification of edges, shapes and facial structure to be applied to my current data, without the need for the network to necessarily learn these itself, rather re-enforce its previous learning with my new data. This should allow for a model which learns faster, requiring less time to train, as well as a more accurate model, due to its additional pre-training. Some information gathered from [9].

2.4.4 CondenseNet:

This is an example of an “Efficient DenseNet using Learned Group Convolutions”[10]. This was designed for mobile use, with the aim of requiring limited computational power by removing connections between layers, aiming to remove redundancies which are not used in the later layers. Due to this, only one tenth of the computation is required compared to the DenseNet (a Deep Convolutional Model where all layers are connected to every other layer, rather than one layer being connected to the next). Comparable to MobileNet (an Inception model approach which focuses on efficiency and power usage rather than accuracy) but requiring less computation. This is done through pruning in the early stages of learning to form a ‘sweet spot’ [10] in between sparsity and regularity. Essentially, group (cluster) sizes are reduced (e.g. $3 \times 3 \rightarrow 1 \times 1$) which results in less computation, but a loss in accuracy, and requirement of more grouping of inputs. The idea of this is to reduce the number of connections during learning, to improve computational speeds and re-enforce routes, at the cost of accuracy.

2.4.5 Further Development Ideas / Key Takeaways:

Model Selection On such a small number of classifications and training examples for each classification, CondenseNet may not be the best option for this project. While it is said to be faster and more optimizable for a device with limited computational ability and power (like the Raspberry Pi I am using), it requires a greater quantity of training data than I have, without crossing between sources, and for the sake of this project, I believe that this may be beyond the scope of what is required for success – but this may change as the project progresses. Xception, while slower, can provide greater accuracy given the training data that I have. This would be good for experimenting on what the greatest achievable accuracy of a model given my training data could be. Not only this, but this model is available in Tflite, so it is possible for the performance to be improved while on the Pi. While not necessary, this is a nice bonus and something worth considering. Documentation on this can be found at <https://keras.io/api/applications/xception/>.

Pre-train Options: To improve the versatility of these AI models, the images that are used for training should be altered to allow for a more versatile classification. Since users will be using this at day and night, it would be beneficial to alter some of the training data (if it isn’t already at day and night) by adjusting the **brightness** to simulate darkness or lower light conditions such as changes in cloud cover or going through a tunnel. ($\pm 15^\circ$)

The camera angle and position can also change depending on how the user sets up the system. While the location of the recordings cannot be changed in post, the **angle** can. This will help to provide some fixability in the model, to still allow for detection of tiredness, even if the set-up is not perpendicular to the driver, like in most of the recordings. ($\pm 20\%$)

Another issue is **zoom**. My hardware, as well as the hardware between the different training data sources, is different. As a result, my inputs may automatically be different based on camera zoom, and how the user positions their face in the focus region, so fluctuations in zoom/crop may be a thing to consider. ($0.9 \rightarrow 1.1$ zoom/crop)

2.4.6 Information Communication:

Webpage Hosting With this project being done in python, for ease of use and versatility of existing libraries for the AI section of this project, it would be best to utilize some existing python libraries for the webpage.

Flask and Alternate Frameworks Flask is a lightweight web framework used for web-based interfaces and API's. In the case of this project, it would be used to host the local webpage. This will allow for offline communication of the drowsiness state.

Tkinter and PyQt provide a graphical use interface (idea for what I need), however, have a far more limited flexibility as well as requiring a physically connected device which will further limit how the project will function and be operated as additional wiring will create clutter as the user does not use something which is found in the vehicle anyway. As a result, neither of these frameworks are likely to be considered in the final result.

Django is a very powerful framework, however, with this comes complexity for a system with such simple routing and visualization requirements.

Another example of another framework to consider is REACT. With past experience in other projects, this has the upside that I understand it already. However, with the project being aimed to be as light as possible to improve efficiency, this client-heavy framework might not be the best for this project.

Overall, the low resource requirements and lightweight nature of this project, as well as the majority of the code base being in python, the obvious choice seems to be Flask to provide a platform-independent web interface.

API A simple way of connecting the webpage to the AI model. There are several different ways of communicating data from the tiredness detection AI to the webpage:

- **Shared File:** The idea is that both programs for AI detection and web hosting are run simultaneously with a common file to relay information between them. However, this could be more compute heavy, something I have expressed repeatedly as being something I want to reduce. Additionally, this can result in race conditions, where there is inconsistency between the file and the actual value. From a hardware point of view, with files being stored on the SD card (with limited numbers of writes) this method can result in shortfalls in the long run.
- **Shared Database:** Using some form of database (e.g. SQLite) can provide additional information as well as filtering. However, because of the simplistic nature of this project, it is likely too costly to do this. While given more time, and a greater fixation on tracking and suggested improvements to inform the user of patterns or issues, this is not in the scope of this project.
- **Use of integrated API:** This is the least expensive option in terms of computing power. You can make use of threading to keep the two lightweight, related programs,

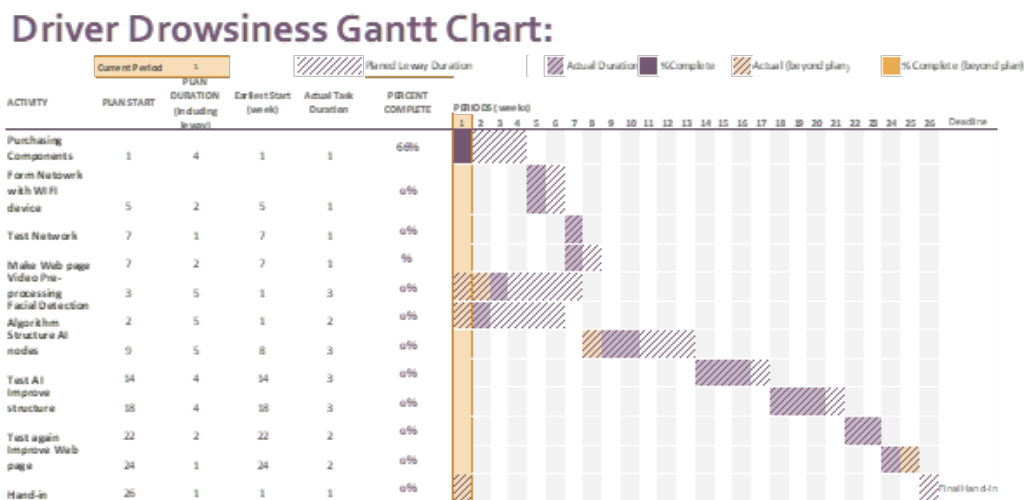
together. This makes it easier to manage and in the event of failure of the AI, the whole program knows (as opposed to being in the dark due to each program being separate).

Therefore, Option 3 (combined server and AI runner with threading) was selected as it minimizes computational overhead, avoids disk I/O bottlenecks, and keeps the architecture simple for this real-time detection application.

3 Plan

In this section, I will outline a brief outline of my intended plan of action, which will likely be reformed later on as other issues become more present during the development process.

3.1 Time Management:



Here week one refers to the starting week (w/c 20.10.2025) and week 26 being the week of the deadline (w/c 20.04.2025). The initial delay in this project is due to shipping of components. Since I do not have any of the components that I need to do this project, I will have to order them, and this takes time. However, in the meantime, I can request permission for datasets as well as gather a better understanding of what attributes can affect a driver's concentration and different potential models that I could use for my lightweight AI component.

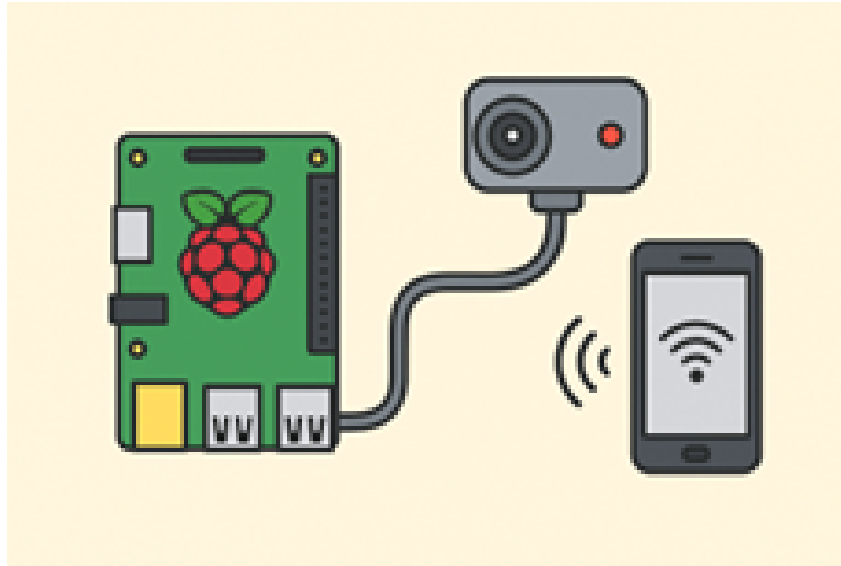
3.2 Connections:

Since raspberry pie 4 is WIFI compatible, I plan to connect the user's own device by this. Therefore, using the pi as a host device for a website that can display data and other information to the user. This is the ideal option as wired displays either require the pi to be:

- Dashboard mounted due to a small wire. This has the potential to fall or move, causing further irritation or distraction for the driver. This is against the aim of the project of keeping the driver focused on the road.
- More wires and additional screens for setting up. Once again, this takes up space in their field of vision and will create added hassle if setting up for different drivers. Most people will likely opt to not use this device, making it a futile construction.

By using the users' own wireless device, you not only remove the added expense of the screen (if released to the commercial market) but also remove clutter. Users are likely to already have phone mounts in their desired location, so why not give the idle device

some added functionality. This will also be optional, if they are already using the device for maps, it will not prevent the system from relaying information or alerting the user.



(ChatGPT generated -> prompt="please make this look nice + drawing of this")

3.3 Component Choices and Reasoning:

Raspberry Pi: For the main processing unit of this system, I have chosen a Raspberry Pi 4B. While this might be over the top in certain aspects, such as:

- Basic Graphics Processor
- 4 USB-A ports
- Ethernet capabilities
- HDMI compatibility
- GPIO Header (40-pin)
- Audio/Video jack

All of these are not necessary for this project specifically; however, I will be using this for a variety of additional projects beyond the scope of my dissertation. I chose model 4 because this has sufficient computation power to process video or images fast enough that it should not hinder the analysis. This should also be able to stay cool (while staying silent due to no fans) while running my intended AI model.

Camera: Initially, I wanted to use a rear dashcam as this was small and would provide the infra-red capabilities I desired for nighttime video, however this required additional wiring which would make this more expensive. In future, I would recommend a smaller camera module, however, for the current constraints and the plug and play capabilities, this is the superior option. These also often support 30fps. This isn't too fast of a frame rate that you are unable to extract and process the data between the frames, as well as not being too slow that you miss vital things such as blinking (which can further be used to detect tiredness).

WIFI Capable Device: This is the area of the project that can have the most flexibility in requirements. I will be making use of several different devices that I own (old phones or tablets), with the intent of users being able to use whatever they have available.

Structure: This project can be broken down into the following areas of consideration:

- **Hardware:** Selection and configuration to allow the support of a real-time driver drowsiness detection system. This includes the use of a Raspberry Pi as the main processing unit, as well as peripheral devices to capture user data required for operation.
- **Data Pre-processing:** To ensure data quality and consistency, before training the models. The handling of missing or corrupted data should be performed, as well as general normalization and clear labeling to allow for an easy workflow.
- **Image Procesing:** This is a core component of the system, enabling facial detection and analysis from still images (frames). This will consist of the following:
 - Facial Feature Extraction
 - Model Development
 - Training
 - Testing

Video Processing: Following the development of single frame processing, it is a good idea to apply this to a series of frames taken from a video input. This will allow for it to give the real-time analysis that is required. This stage will consist of:

- Extracting frames for reference
- Facial Feature Extraction
- Model development
- Training
- Testing

Following the development of the AI model, the next stage of the project is to display these findings and apply warning or other safety measures to the user.

Networking: Connecting to the user's device. Providing a connection to host a webpage on, and later allow for alerts and information to be passed

Webpage: Develop a simple webpage to display system outputs and status information.

Hosting: Host the webpage locally on the Raspberry Pi in a display-only configuration. This will help to ensure that the user does not interact with the system from within the vehicle. This will help to ensure safe operation in a vehicle setting.

Mobile Device Connectivity: Connect the user mobile device to the Raspberry Pi via local network connection to access the hosted webpage

Additional: To ensure safety and conform to laws, disable actions from the webpage (if any are sued) while in motion.

4 Datasets

For this project, I have used several different data sets from varying sources around the globe in order to try and train a more versatile system. From Turkey to China, all of these sources are publicly available and will be noted, referenced and explained in this section. Please note that additional references will be made through this document. I will outline where the datasets can be found, steps to access it for yourself as well as legal outlines that I must abide by in order to use them.

4.1 SIVAS-UNIVERSITY-OF-SCIENCE-AND-TECHNOLOGY-DRIVER-DROWSINESS-DATASET:

Collected by a Turkish University, this data set contains 19 drivers (3 female and 16 male) who recorded themselves using their phones in a variety of positions. These are in .mp4 formats. [3] Their GitHub can be found at: <https://github.com/EsraKavalci/Sivas-University-of-Science-and-Technology-Driver-Drowsiness-Dataset>

4.2 VICOMTECH:

This organization has many different datasets available such as Body pose estimation and Hands-on-wheel detection. Following my research into fatigue identifiers. I first found out about this dataset through FatigueView [5] where it was referenced as an external data set. This dataset once again has a variety of users, this time with an average age of 30. IT also outlines additional features to consider, such as glasses wearers and other driving activities such as drinking water, on the phone and other actions which may be useful to better understand how drivers act differently when fatigued. Their Page: <https://dmd.vicomtech.org/> [4]

4.3 FATIGUEVIEW:

This is not only a dataset, but also an article on how tiredness affects drivers and what identifiers exist to help identify fatigue. This uses data collected from several different people, some many different backgrounds. This data set not only contains videos of people who are already tired but also shows how people can change from awake to drowsy as time goes on. This is done through a simulated driving environment with many cameras at varying angles to help capture the most versatile data. This will be useful, as the varying camera angles will allow the model to be more forgiving to change, allowing for an individua to place the camera in the best location for them in their own car. Their article: <https://ieeexplore.ieee.org/document/9931532/figures#figures> [5]

4.4 BINA University:

A dataset collected by a university in Indonesia, the idea of the study was to “hypothesizes that drowsiness can be accurately detected in real-time through computer vision analysis” [7]. This was collected using a Raspberry Pi 5 and a camera module, taking frames at a rate of 30 FPS. This dataset takes more of a face on positioning. This will allow for more accurate extraction of features for testing as turning of the head is less likely to result in the loss of information (where other datasets with more of a bias to one side

will lose facially features when they turn away – the same change has a more dramatic effect). Furthermore, this dataset set also contains night recordings. This is important as you are:

- likely to be more tired at night
- contrasts in images and the visibility of attributes is likely to change (due to reduced lighting) so it is important to take this into account

<https://data.mendeley.com/datasets/chvz7vh2dc/1> [7]

4.5 DATASET POSITIVES AND NEGATIVES:

Camera Location: By using a variety of different camera angles, when training the model, it will be more lenient to change. Fixed locations can result in the data being more tailored, and so the AI model will be more tailored. This can influence reliability as it can result in the model being more specialized. Changes to location could potentially have a large impact on the output and its reliability. Varying countries can also have the same effect. Different countries drive on different sides of the road and therefore drive on different sides of the car.

Varying User Groups: Like location, varying users is key. Different ethnicities and languages can vastly improve data reliability. While some individuals are more expressive and others harder to identify features, this can help improve the versatility of the extracted data. While not something I would have initially considered, language spoken can play an effective role. Some languages are more articulatory and expressive so require more facial movements, which may be construed as yawning or signs of tiredness (especially if audio isn't considered).

Varying Hardware: While my project intends to use consistent hardware throughout, the variation in hardware used during data collection across different datasets provides a degree of flexibility. Since my hardware doesn't exactly match that of one or more datasets, having access to multiple datasets helps mitigate potential inaccuracies that could arise when testing on fixed data versus real-world usage scenarios.

varying Time: With some data sets spanning both night and day, there will be ranges in clarity and lighting. It can be a safe assumption that people (in most instances) are more tired at night; be this due to a long day or your circadian rhythm (your body's internal clock). This may also vary the number of 'Tired' events in a given period, simply due to time of capture.

Bais: With less women taking part in the study (across all datasets) it is likely that it will be less accurate at detecting their tiredness. While the general indicators are the same, additional features such as longer hair which can distort imaging may play a part. Additionally, glasses (while represented in the dataset) might influence facial detection and lead to false results. This is something I will have to consider later. Other areas such as eye shape and size varying through individuals might make certain groups of people more difficult to identify tiredness, while others are more prone to different patterns for being tired. Time of day will also be a factor. It is easier to see people in the day (when

they are more likely to not be tired). As a result, determining tiredness might not be as straight forward as the less clear data (relating most to the area I want to identify and train the model for) will be harder to extract information for. Not only this, with some datasets being collected from nighttime workers, they might be tired during the day, which will reduce the effectiveness of time within the data.

5 Development Checklist

In this section, I will be noting any tasks that are required to complete the project. While some smaller sections that posed no issues may be glossed over and only noted in the "Development" section, most of the step-by-step stages will be mentioned as a check list for someone to follow if they want to create this project for themselves.

Table 1: Project Task Overview

Task Ref	Task	Date Started	Date Completed	Status	Comments
T1	Purchase Components	–	29/10/2025	Complete	
T2	Face Identification	–	04/11/2025	Complete	In pre-captured images
T3	Taking frames	10/11/2025	11/11/2025	Complete	Able to take an image from the camera (laptop only)
T4	Extracting Features	20/10/2025	23/11/2025	Complete	Using MediaPipe to identify features – not modular at this point
T5	Validate Data	07/11/2025	08/11/2025	Complete	Compare overlay on image to ensure correct alignment
T6	Train Simple AI	08/11/2025	18/11/2025	Complete	Not compatible with Pi yet; required additional preprocessing
T7	AI model on Pi	23/10/2025	25/11/2025	Complete	Lightweight model from Bina dataset; works only on individual frames
T8	AI model for Video testing	01/12/2025	03/12/2025	Abandoned	Inaccurate; buffer issues with model
T9	Data pre-processing	10/12/2025	19/12/2025	Complete	Prepared dataset for advanced model
T10	Advanced model training	20/12/2025	24/12/2025	Complete	
T11	Advanced model validation	26/12/2025	27/12/2025	Complete	Tested on validation dataset
T12	Advanced model testing (real world)	22/01/2026	22/01/2026	Complete	Tested with camera input
T13	Testing on Pi	–	–	Complete	tf lite model works, but required a reduction in tf_runtime environment due to library conflicts

Task Ref	Task	Date Started	Date Completed	Status	Comments
T14	Set up connection between devices	01/02/2026	03/02/2026	Complete	Connection established; webpage transmitted
T15	Form basic webpage	–	–		
T16	Display webpage on device from Pi	–	–		
T17	API testing	–	–		
T18	Linking API and AI model	–	–		

6 Development

6.1 Setup:

Python Versions On my laptop (the main device for coding) I am making use of Python 3.10.5. This is because, mediapipe is not currently supported by the later versions that are available(at this time 3/1/25 it is supported up to 3.11.0). While the version that is installed by default on the pi is 3.13.5, I will be reducing this down to the same 3.10.5 to ensure all libraries work. To ensure there are no conflict, I will be using a virtual environment to run my python scripts in the correct version.

PI OS Setup I am using Debian with desktop for my Pi OS version as this is the most UpToDate (and recommended for my device). I am using the desktop version as this will allow me to more easily change code rather than using a straight Command Line interface, however, it would be possible, and I could ssh into the Pi to execute code from there.

Initially, I did try the command-line approach, however, issues in the connection timing out and for the sake of convenience, I thought it was best to make use of a desktop OS version.

Face Detection I am using the libraries OpenCV (Open-Source Computer Vision) and Mediapipe to detect the face and extract data. OpenCV allows me to open the camera, read a frame and detect the existence of faces. After this, I am making use of a Mediapipe library to extract facial details. Using some of the built in features these libraries provide, I then overall a triangular mesh which I can use to extract key points on the face, such as:

- Eye openness
- Mouth position

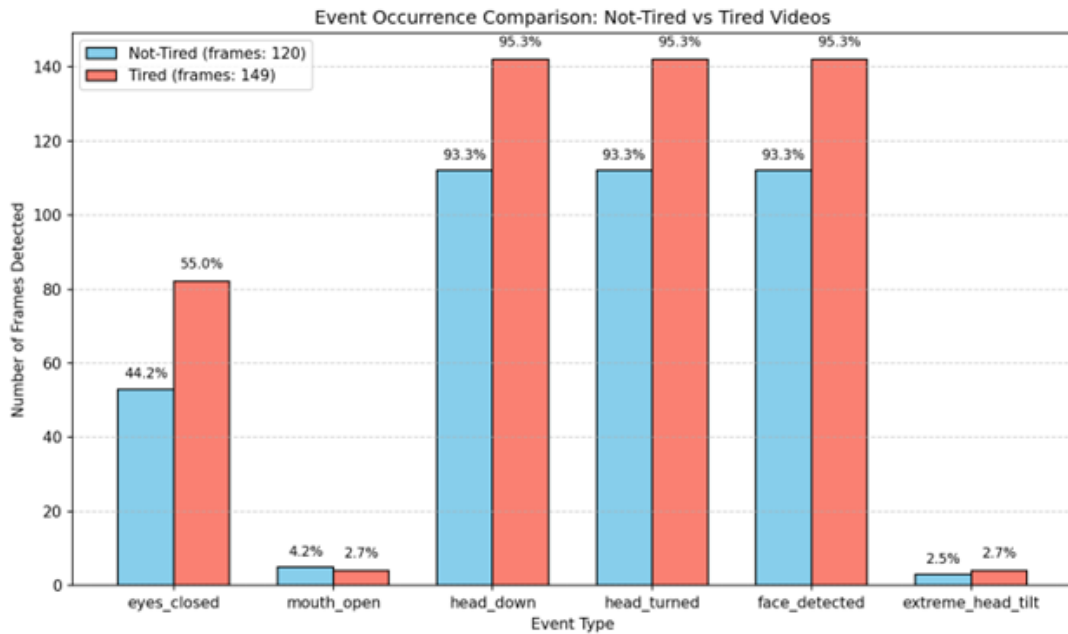
- Nose and chin locations to determine orientation / head tilt

Together, I can take video and images, and then extract the information required for me to determine drowsiness.

6.2 Data Validation

This section is designed to break down the spread of events in a data set and ensure that there is a difference in the data, so it is viable for training some form of AI.

SUST Dataset This refers to the ivas-University-of-Science-and-Technology-Driver-Drowsiness-Dataset [3] This shows an example of 5 videos for each classification (chosen at random). This is using a 2fps sample rate.

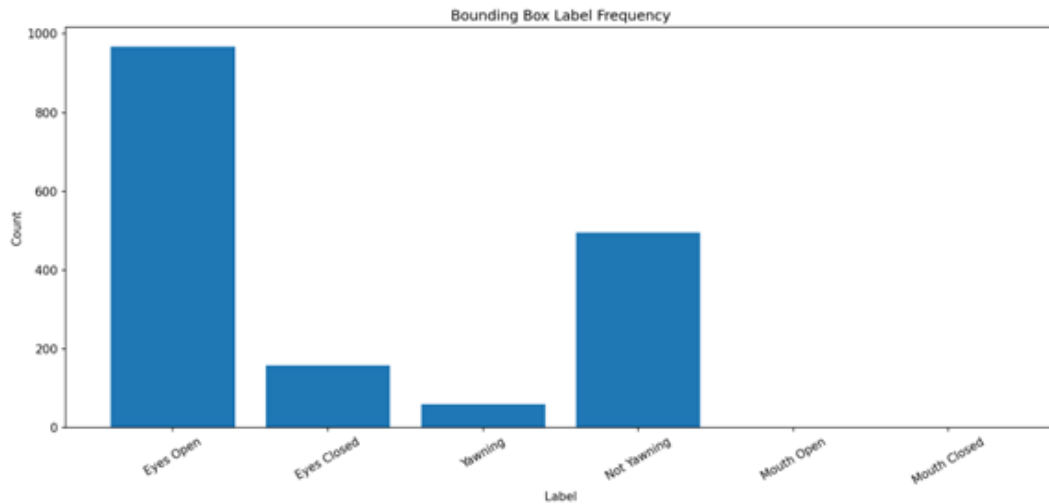


As you can see, there is a distinct difference between the 2 datasets, and so this is viable for development of an AI model. However, surprisingly (could be just from the chosen videos) that there seems to be reduced mouth movement when tired. This was not an expected result!

Something else that caught my eye was the number of **extreme_head_tilt** events. By playing around with threshold values, I was able to determine that while tired, you tilt your head more (physically) than when awake.

However, following on from this, I formed a lightweight model where I passed in a selection of 5/6 videos of each classification and attempted to see if these events alone were enough to form a valid model. This was a failure. This is because despite there being frames which contain these tiredness events, most of the video was populated with frames which showed no attributes of tiredness (despite tiredness events, a majority of the 'normal' frames were classified as tiredness). This resulted in a poor model which had very low confidence in its classification (52%). Therefore, this is not a viable strategy.

Bina Nusantara University This dataset (as discussed earlier) is a collection of images, which are labeled with “eyes closed”, “eyes open”, “yawning” and “not yawning”. These labels are purely from the pre-made labels file that comes with the data; however,



I wanted to be sure that there was a range of instances for both the tired and not tired counterparts. This suggests that there is enough variation in the dataset to potentially form an AI model.

6.3 Simple AI Models

6.3.1 Libraries

TensorFlow Easy to use and customize. One thing I have done to help improve the computability of the project was do most of the training and coding on my laptop, then importing the project from my Git Repository onto my pi, allowing me to train models and test more easily, rather than trying to do it directly on the pi. Using TensorFlow, I am easily able to save models as ‘.h5’ file and then run them later.

TensorFlow Lite For ease of use and in the hope of improving running time, I will be converting my standard ‘.h5’ models into ‘.tflite’ models. This is because general tf models are designed for training as well as inference, while tflite models are aimed more at just passing data through them. In the case of a Pi, this more specified computation will help to improve the return rate of the model/service, allowing me to run it more in a given time. This is done by compressing the models, reducing the required RAM and using a C++ runtime as opposed to the normal (more cumbersome) python.

6.3.2 Simple AI Model

Based on my initial findings of what constitutes tiredness, I thought it best to extract the features that determine tiredness and input their ratio values into a simple CNN model.

The model that I have used so far is comprised of 5 inputs [left eye aspect ratio, right eye aspect ratio, mouth aspect ratio, pitch, yaw] extracted from the frame. This is then assed into a convolutional model consisting of 2 hidden layers of size 32 and 16 respectively. This uses the ‘Adam’ (Adaptive Moment Estimation) optimiser method which adjusts

weights combining the benefits of momentum and RMSProp. On each layer the model makes use of ReLU (Rectified Linear Unit) as an activation function, allowing positive values to pass through and negative ones to change to 0.

From this simple model, I was able to produce deduce tiredness at an accuracy of 74.5%, based on frames extracted from the Bina Nusantara University dataset. I believe this is because the classification of eyes open compared to eyes closed, where some examples of training data (as well as test data) were miss labelled from the source (so it is worth while to change these like I did to ensure the data is classified correctly for training). This improved accuracy slightly to just over 76%.

Following this, I thought it was a good idea to test its compatibility on the Pi and use myself as a test subject. Initially, I thought this was working brilliantly, as closing my eyes or imitating a yawn would result in the classification of Tired (with a similar amount of accuracy to on the testing data). However, as my testing became more extensive as I played around with varying facial expressions (to test the limits of the model) as well as different orientations (looking to the left or right to simulate varying camera positions), I noticed that my tilting your head to the left (moving your ear down towards your shoulder) that you would be classified as tired, no matter the rest of your facial expressions. This is something that was completely overlooked in the testing data and implies there is a subtle trend in the data that your head tilted left is a sign of being tired. This is not true for the real world and could be down to camera position.

My initial thought was to retrain the model as it may be that that classifier (through that run of training) has gotten to a local maximum (locally high bias) and been unable to recover. However, despite other iterations, I was unable to produce a more accurate model, fluctuating between 68% - 76% accuracy on training data, with no more luck on testing on myself and a webcam.

From here, I thought it best to experiment with a different model to see what the upper bound in obtainable accuracy was.

6.3.3 Inception Model

This will be used to determine an upper-bound on the potential accuracy a model can give. With its primary purpose, to identify images, this is ideal. As stated in the research section of this document, I will be proceeding with the Xception Model to classify images. The initial model I used was pre-trained on the ImageNet dataset. This will allow for the easy identification of features (e.g. lines and shapes) without needing for the model to learn them from scratch, which should help to improve learning rate and require less data inputs to create a fine-tuned model. From here, I trained it on the Bina University [7] dataset as it was composed of just images and contained a label file which would make classification easier.

Issues: Following the training of the model, the outputs as to classifications were in the same order as they were in the labels file. As an oversight on my part, I did not translate these labels (in the file) to English as (from the overview of the project) I could do it later. However, during my testing, I found some of the predicted labels were not lining up with my perception of the images. For example, I thought an image was someone with their eyes closed, but the labels said otherwise. I went through I found that the dataset only had

3 classifiers in the labels, not the 4 that I had been told. From what I could understand, yawning and not yawning were translated properly, however the eyes open/closed labels were substituted in for a neutral face label, which has the same translation as eyes open (when converted to English).

Solution: To fix this issue, I wrote a short section of code (based on my initial feature detection process from the data validation section, to turn these classifications into valid labels (only for eye openness). Following this, I then went through each image, which had changed, and verified the labeling. Following this, I then re-trained the model on these new labels. Then I tested it on the validation dataset, resulting in 131/137 (95.6% accuracy) images being correctly classified. With those which were wrongly classified, I went through and had a look at where the model was wrong. In 4 out of the 6 total images which expressed false-positives for eyes closed, I could understand why the model had perceived the people as having their eyes closed. In the other 2 examples, one person was wearing sunglasses (which unsurprisingly) caused an issue, with only 1 being truly wrong, classifying as eyes closed, when they were open, as well as yawning when their mouth was only slightly open.

Based on these findings, I then wanted to check how the model worked on me as a use-case. To my disappointment, it was not as successful as the validation would suggest with only around a 70% success rate based on my attempts to fake tiredness. While I understand that my acting abilities are not perfect, I was very surprised to find that I was not able to get a result in a similar ballpark, given that fact my expressions were likely over-exaggerated. Additionally, when looking back through my code, in my attempt to push a working model out before the exam season began, I have noticed that I had not altered or included the variations to input data, like I had in the Simple Convolutional AI models. As expected, after adding in the variations to:

- Brightness
- Contrast
- Saturation
- Flipping the image

,the accuracy of the model has decreased (relative to the testing set), with the hope that it is more versatile for a wider variety of end implementations. With a 75.91% accuracy (104/137) on the Bina University dataset resulting in correct predictions, this was more of a detriment than I predicted.

6.3.4 Dataset Resolution

While the previous model was reasonably accurate on determining if the user had their eyes open/closed or if you were yawning or not, it would require an additional threshold to determine tiredness from this. In my earlier model versions, I determined the classifications of eyes closed or yawning as a positive case of tiredness. However, with the Xception model taking in image inputs, I wanted to experiment with opinionated learning. This would allow me to define the dataset based on my opinion of someone looking tired and considering a larger array of features such as:

- Eyes open (but not fully), implying they are drained
- Bags under the eyes
- Mouth positions for talking vs yawning

,with this in mind, I decided to go through manually re-classifying the data and re-train several models on this. One of the things to consider when doing this is opinion. Since I am classifying the data, my opinion may be different to someone else, which can lead to inconsistencies based on person classifying because of a larger array of external factors. With this being an opinion and not necessarily a definitive solution yes or no, there was a slight reduction in accuracy to 73.72% (only a 2.19% loss compared to the previous model which used definitive classifications); far better than I could have hoped, for with an 85.33% accuracy in detecting ‘alertness’.

Personal Testing: Following my pervious tests on myself, I found that I would try and remove personal bias to a favorable model by formulating a quantitative dataset that I can test against. While only consisting of 100 images and a fixed environment, I wanted to play around with varying expressions to validate functionality. Despite using a webcam and in my room with no natural light and unlikely camera placement (vastly different from the actual use case), I was still able to obtain a 63.27% accuracy in detecting ‘tiredness’. From the results, however, all errors were false positives for alertness (mistaking the user for being alert compared to tired).

explain what you might change to help rectify this issue and why it may be caused... maybe more testing data that is more real-world

7 connectivity

In the following section I will discuss how the classification will be presented to the user.

Overview As previously outlined in the design section, I would like for the information to be displayed as a webpage hosted on the Pi. This will require several components:

- Web page: for the actual displaying of information
- API: for the sending and receiving of the information from the AI model to the user
- WIFI Hosting: to allow the user to connect to the device in a dynamic setting rather than using a common network which may or may not exist in a movable setting such as a car

7.1 Back-End:

Network Architecture and Hosting By utilizing Raspberry Pi OS (Desktop), I can easily toggle between “Hotspot Mode” for device testing and “WI-FI Client Mode” for pulling updates via Git. This flexibility of dual purpose drastically accelerated the development and testing cycle of this section of the project, with the ease of not needing to type commands or form a toggle file.

Initial Flask Implementation To validate the connection, I developed a light-weight Flask application to serve a basic index.html file. This contained no information, images or other resources, just text. This confirmed that devices could successfully connect to the Pi’s hotspot and access the web server by inputting the Pi’s IP address. This application was constructed to specifically use port 80 (HTTP), removing the need to define the connection protocols when handling IP addresses (Such as when changing the hostname)

Local DNS Resolution To provide a seamless user experience, I wanted to remove the IP-based navigation and instead navigate with a custom hostname. To do this I used ‘dnsmasq’. Using this, I configured the domain “drowsiness” directly to the local IP address for the application.

Performance Optimisers (DNS Upstream) Since the system operates as an offline connection, with no external internet access, I disabled recursive DNS lookups¹. This improves the performance by preventing the Pi from attempting to contact upstream servers for un-known queries, eliminating the latency caused by waiting for timeout responses. However, with this in mind, it is important that this change is reverted when connecting to the wider internet as this can prevent you from accessing resources such as the GitHub repository. To revert simply remove “no-resolv” from /etc/dnsmasq.conf.

remmeber this

¹the process where a device tries to ask other (more senior) servers for corresponding IP addresses for addresses outside of its local scope/knowledge

```

no-resolv
no-poll
interface=wlan0
address=/drowsiness/10.42.0.1
dhcp-range=10.42.0.2,10.42.0.254,255.255.255.0,24h
dhcp-option=3,10.42.0.1
dhcp-option=6,10.42.0.1
# Configuration file for dnsmasq.

```

Listing 1: dnsmasq Configuration

Table 2: Explanation of dnsmasq Configuration Parameters

Configuration Line	Functional Explanation
<code>no-resolv</code>	Tells the system not to look at the system's <code>/etc/resolv.conf</code> file, preventing it from using the ISP's upstream DNS servers.
<code>no-poll</code>	Stops the service from checking for changes in the upstream DNS configuration files. While not strictly necessary, it helps maintain system cleanliness.
<code>interface=wlan0</code>	Restricts the service to listen only for requests on the Wi-Fi interface. This allows the Ethernet connection to remain independent for interacting with the GitHub repository.
<code>address=/drowsiness/10.42.0.1</code>	Acts as a local DNS redirect for the 'drowsiness' domain, pointing it directly to the application's IP.
<code>dhcp-range=...,24h</code>	Defines the pool of valid IP addresses a connecting device can be assigned. This lease lasts for 24 hours.
<code>dhcp-option=3,10.42.0.1</code>	Option 3 (Default Gateway): Tells connected devices to send their internet-bound traffic to this machine (10.42.0.1).
<code>dhcp-option=6,10.42.0.1</code>	Option 6 (DNS Server): Instructs connected devices to use 10.42.0.1 as their primary DNS server.

Multi-User Scalability To allow for multiple devices to connect at once, I configured a /24 subnet mask to manage IP assignments. While the Raspberry Pi's hardware typically supports 5-10 devices connected concurrently, this ensures that:

- Multiple passengers can connect without causing IP conflicts
- The driver's connection remains stable, ensuring that safety critical data is always accessible, even if other devices accidentally "auto-join" the network

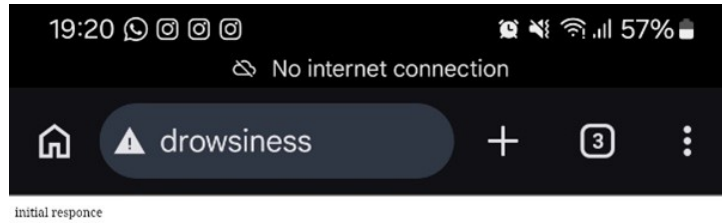


Figure 2: This is a Simple example of how a webpage URL can be inputted to make it easier for the user to find the resource. In this example, while ‘drowsiness’ worked on one browser, ‘drowsiness/’ had to be used on another as it assumes it is a search not a URL.

Captive Portal One of the other features that would be ideal is the use of a Captive Portal. Often, when connecting to public or guest Wi-Fi networks (such as a coffee shop or cinema) you must go through a page to accept their terms of conditions and potentially log-in. While I will not be using a log-in system, agreeing to terms and notifying the user of issues using the phone may cause would be a good moral stead, as well as it then easily allowing the user to interface/view the webpage after acknowledging the potential risk. This will be done by using DNS hijacking.

more about this

API As previously mentioned, I have opted for an integrated API and AI model solution, where the model work to use in-memory state management rather than a database or file; the main reasons being to reduce complexity by avoiding I/O overhead provided from databases and avoid race conditions.

Structure The API file will be broken into 3 sections for simplicity and ease of use:

- **Flask Initialization** Since this file will eventually be run on start-up of the device, rather than a manual start up (so that the system is up and functional for ease of use), I have decided to group the initialization of the webpage to this file. While this could be separated out, the simplistic nature allows for a cheap and easy way of ensuring everything is functional. Additionally, with it being an operation which calls other processes, it also allows for other areas of the project to load in the mean-time (e.g. load the AI model from memory). Furthermore, this asynchronous initialization... **finish**
- **Prediction Generation** This will allow for both the model’s predictions and the calling function to have access to the same variables. By using threading, I can continuously run the model without interfering with other procedure calls, allowing for seamless and continual integration. This is especially important when considering the multi-user capabilities. If the webpages are not synced (highly likely), API calls from each independent version of the webpage would otherwise cause the AI model to stop capturing data while the calls are answered. This would result in fewer frames being captured and processed, increasing latency to this critical safety system. While the use of threading can invoke race conditions by each thread trying to both interact with the same variable, ‘thread.Lock()’ is used to freeze a variable

during assignment or reading to only allow one process to use this variable at a time.

- **Web Call Processing** Rather than refer to a direct IP address (which may vary between devices and can be dependent on other connections), this API call uses relative URL routing to easily find the correct API. This is potentially less secure, however, with the final aim being to remove all other processes and files and only have the API, webpage and AI model on the device, here is little more that a potential attacker could do. In an ideal world, with the correct hardware which is tailored specifically to this project, forming a more market focused product, the scope of what could be done would be reduced and **finish this off**

7.2 Front-End:

Webpage Design The primary objective of the front-end interface is to communicate the user's predicted alertness efficiently while maintaining a discrete aesthetic. To ensure the interface is non-intrusive during general use – regardless of ambient lighting – a permanent dark-mode theme will be implemented. This will minimize user distraction and reduce eye strain during both day and night time use.

Design Ideas This section will give some examples of potential ideas. Some of these ideas will not make it into the final product, but may wish to be considered in other related projects or in the event this project is done again. **insert sketches and ideas**

end with one idea as a final choice and explain each idea and its pro/cons

Integration Following the construction of a basic frontend, I wanted to see how it would look as a changing display – using random variable generation to get a better view of the system. To do this, I simply used the 'random' function to generate appropriate (possible) values and store them in the shared variables to be sent upon API call.

Insert Image

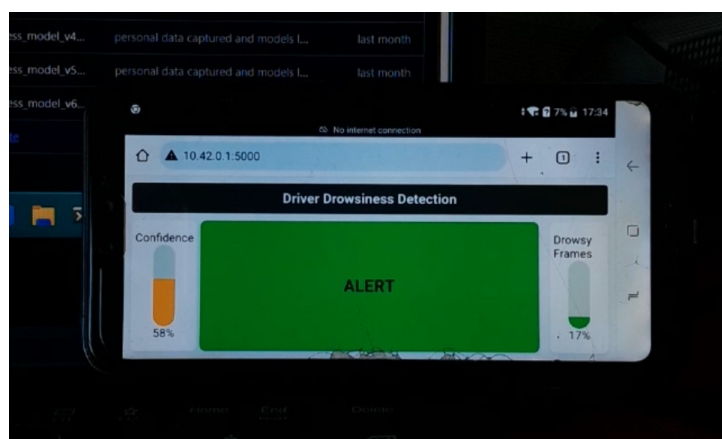


Figure 3: Example of a simple webpage layout.

8 Evaluation

References

- [1] Brake. (2026). “Driver Fatigue Knowledge Centre.” <https://www.brake.org.uk/get-involved/take-action/mybrake/knowledge-centre/driver-fatigue>
- [2] Brake. (2026). “UK Road Safety Knowledge Centre.” <https://www.brake.org.uk/get-involved/take-action/mybrake/knowledge-centre/uk-road-safety>
- [3] Kavalci Yilmaz, E., and Akcayol, M. A. (2022). “SUST-DDD: A Real-Drive Dataset for Driver Drowsiness Detection.” *Proceedings of the 31st FRUCT Conference*, 31:416–21. doi: 10.5281/zenodo.6519933.
- [4] Ortega, J. D., et al. (2020). “DMD: A Large-Scale Multi-modal Driver Monitoring Dataset for Attention and Alertness Analysis.” *Computer Vision — ECCV 2020 Workshops*, pp. 387–405. Springer International Publishing.
- [5] Yang, C., Yang, Z., Li, W., and See, J. (2023). “FatigueView: A Multi-Camera Video Dataset for Vision-Based Drowsiness Detection.” *IEEE Transactions on Intelligent Transportation Systems*, vol. 24, no. 1, pp. 233–246. doi: 10.1109/TITS.2022.3216017.
- [6] UK Government. (2024). “Using a mobile phone while driving.” <https://www.gov.uk/using-mobile-phones-when-driving-the-law>
- [7] Isworo, N., and Liawatimena, S. (2025). “Annotated Drowsiness Detection Dataset Captured Using Raspberry Pi 5.” *Mendeley Data*, V1. doi: 10.17632/chvz7vh2dc.1
- [8] Szegedy, C., et al. (2015). “Going deeper with convolutions.” *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1–9.
- [9] Chollet, F. (2017). “Xception: Deep Learning with Depthwise Separable Convolutions.” *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1800–1807.
- [10] Huang, G., et al. (2018). “CondenseNet: An Efficient DenseNet Using Learned Group Convolutions.” *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 2752–2761.